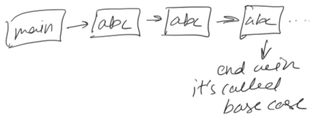
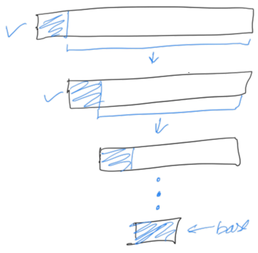
Recursion

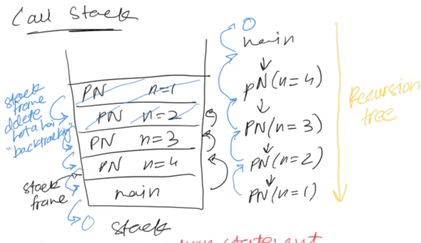
↳ fnx calls itself



everything we do with for loops we can do with recursion



Call Stacks



after n=1 the return statement sends us to the previous stack frame and we delete the current frame this process is called backtracking end main the stack will be empty

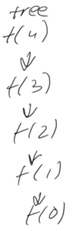
Time complexity

```
int factorial (int n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial (n-1);
}
```

2 ways to find TC

1. Recurrence relation
2. TC = total no of * work in rec calls each calls

Rec tree



n = 4

for n we calling n+1 time

n ~ n calls

n x work in each call — constant work

O(n x k)

⇒ O(n)

Space complexity

we consider the auxiliary space (extra variables) + call stack

SC = depth of rec tree * mem in each cell

⇓

SC = height of call stack * mem in each call

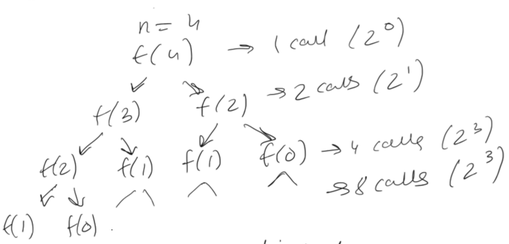
Fibonacci

$t_n = t_{n-1} + t_{n-2}$

0, 1, 1, 2, 3, 5, 8, 13, ⋯

find n^{th} term

```
int fib(int n) {
    if (n == 0 || n == 1)
        return n
    return fib(n-1) + fib(n-2)
}
```



TC = total calls * wd in each

$= \left[\frac{2^0 + 2^1 + 2^2 + \cdots 2^{n-1}}{2^0 * (2^n - 1)/2}\right] * k$

$\Rightarrow 2^n - 1$

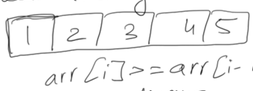
$= O(2^n)$

SC = Depth of RT * ver

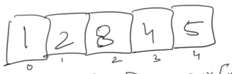
$= n * 1$

$\Rightarrow O(n)$

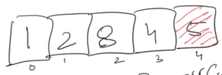
Check if array sorted



arr[i] >= arr[i-1] always



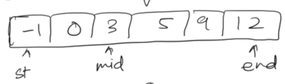
arr[n-1] ≥ arr[n-2] ✓



arr[n-1] ≥ arr[n-2] ✗

```
isSorted (arr, n) {
    if (n == 0 || n == 1) return true
    return arr[n-1] >= arr[n-2]
        && isSorted (arr, n-1)
}
```

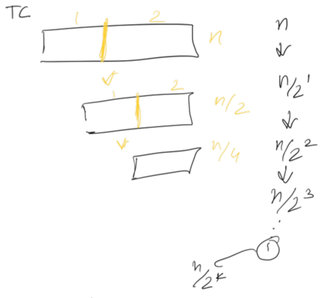
Recursive Binary search



tar = 9

```
mid = st + (end - st) / 2
if (a[mid] == target) return mid
if (a[mid] <= target)
    return bs(arr, mid+1, end, tar)
else if (a[mid] >= target)
    return bs(arr, st, mid-1, tar)
```

TC



$\frac{n}{2^k} = 1$

$n = 2^k$

$\log_2 n = k$

$TC = \log_2 n * O(1)$

$= O(\log n)$

$SC = O(\log n)$

Print all subsets

arr = {1, 2, 3}

{1} {2} {3}

{1,2} {2,3} {1,3}

{1, 2, 3}

{} = 8

n elements ⇒ 2^n

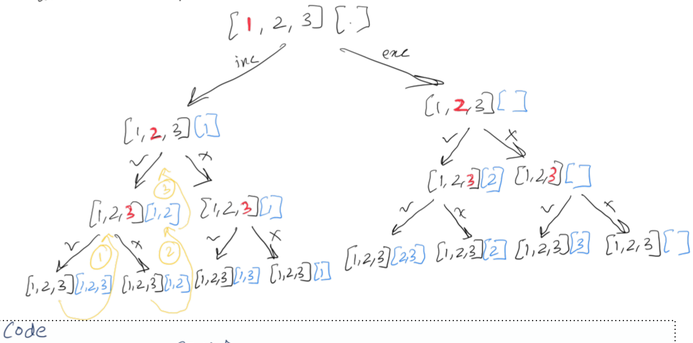
each element has a choice whether it wants to be included or excluded

so let's start with

arr {1, 2, 3} &

an empty arr []

and we will keep storing in empty arr



Code

```
void ps (arr[], ans[], i) {
    if (i == arr.size())
        print the ans & return
    ans.push_back (arr[i]) // inclusion step, adds the arr in ans [1, 2, 3]
    ps (arr, ans, i+1) // goes to previous level (step ①)
    ans.pop_back() // (IMP) the actual backtracking step, this will remove the last element and will give [1, 2]. now we can move to the next step
    ps (arr, ans, i+1) // exclusion step (step ②)
}
```

we do this step because we now want to do exclusion so we can't keep that element so we pop it out

TC = total calls * work done in each

$= (2^n \times n) * k$

$= O(2^n \times n)$